

Proofline: method note

RealPage challenge, Rental Housing Law Navigator · Hack-Nation 7th Global AI Hackathon · as of 2026-10-01 · Not legal advice

proofline-opal.vercel.app
github.com/usv240/proofline

What it does

Reads the supplied housing-law corpus, turns each rule into a structured record with a machine-testable coverage condition and an exact quote, resolves any address to its legal city, tests every rule against the building on a chosen date, and reports which addresses each law change affects. Every answer cites its source, its retrieval date and the law's own words; when coverage depends on a fact the records lack, the answer is "unknown" with the deciding fact named.

Pipeline (Module A)

1. **Read** (Claude Opus 5.5, structured outputs): one call per document returns rule records in the official schema plus a coverage test over a closed set of facts (units, year built, building type, owner occupancy, subsidy, tenancy length).
2. **Check quotes** (code): every quote must be found word for word in the source; the stored span is copied from the source, never from the model. No quote, no rule.
3. **Second check** (independent model pass): tries to refute each record from the document alone; refuted records are dropped, partial ones corrected with lower confidence.
4. **Reconcile and assemble** (model grouping, deterministic merge): the same law found in several documents becomes one record; status is computed from lifecycle dates (enacted, effective, struck), never typed; conflicting published dates are kept and flagged.
5. **Re-anchor**: rules first read from link-only pages are re-cited to official corpus text wherever it states the same rule (91% of "applies" answers quote the supplied corpus).

Address lookup (Module B)

All 500 addresses resolved to their incorporated place (477 by US Census Geocoder coordinates, 23 by mailing city). Unit counts come from the assessor column, NJ MOD-IV descriptions ("3S-F-D-6U" = 6) or official use bands. Coverage uses Kleene three-valued logic: a missing fact gives "unknown", never "no"; a building finished in a certificate-of-occupancy cutoff year is "unknown". A state rule that yields to local law is "superseded" where the local rule applies; tenant-level conditions (tenancy length) are stated on the answer rather than blocking building coverage.

Change tracking (Module C)

As-of evaluation from lifecycle dates. T1 250 CA addresses; T2 Jersey City 50 and Hoboken 40, Newark 0; T3 140 NJ addresses with 90 conflict flags; T4 110 MA addresses as pending; T5 empty. New laws run through the same pipeline with one command; Bayonne, NJ was added live from its official ordinance to show this (kept out of the submission files).

Results and validation

- 74 rules in the 13 places in scope, every one with its quote found word for word in its source; 83 of 87 manifest documents read.
- Negative control: **0 "applies" answers in 2,010 checks** where the right answer is no rule, failed, or not law yet.
- Baseline (same model, BM25 search over the same corpus): claimed an in-force rule in 2 of 33 no-rule questions and gave 1 quote(s) not in the sources; Proofline 0 and 0.
- 32 automated checks, including golden checks taken only from the organizers' statements (brief example, README cutoffs, T1 to T5). They caught two real bugs during the event, both fixed. `npm run verify` re-checks the hash-chained audit log, every quote, and that the engine reproduces lookups.json.

Responsible design and limits

Not legal advice on every page, API response and export. Pre-Flight answers only allowed, not allowed or needs a person and never suggests ways around a rule. New or changed laws are proposals until a person approves them. Measured limits: about 14% of answers are "unknown" because public records omit year built (San Diego, Berkeley) or unit counts (Berkeley, Jersey City, Newark, Boston); 9% of "applies" answers quote link-only pages (Hoboken, Newark, the Jersey City ban, the LA source-of-income article), labeled as such; owner facts are never in public data.

Scalability

A new place or law is one command (`npm run new-law`): read, check, merge, test, publish, verify. The engine is shared by the pipeline, the web app (it runs in the browser, no model calls at answer time), the public API with keys, and an MCP server.